

Project implementation

I think this is the strongest approach for a final-year project. It strikes a good balance between research, implementation effort, and the likelihood that you'll have a fully working demo. It also matches the methodology you've described in your PPT while avoiding the risk of spending months trying to reproduce entire research papers.

I'd refine it slightly so that each part has a clear purpose.

Module	Implementation	Source
Video upload	Build ourselves	Custom
Audio extraction	FFmpeg / MoviePy	Open source
Face detection	MediaPipe Face Detection	Pretrained
Face tracking	MediaPipe + SORT/ByteTrack	Open source
Lip crop extraction	OpenCV	Custom
Visual feature extraction	Pretrained visual encoder	MuSE / Looking to Listen
Audio feature extraction	STFT + Mel Spectrogram	PyTorch/Torchaudio
Speech separation	Open-source AV model	MuSE or Looking to Listen
Missing visual cue detection	Custom confidence-based detector	Inspired by RAVSS
Audio-only fallback	Existing audio separation model	Conv-TasNet / SepFormer
Speaker selection UI	Build ourselves	Custom
Bounding boxes	Build ourselves	OpenCV
Playback	Build ourselves	Custom

What should be implemented by your team

This is where your engineering contribution lies:

- Complete processing pipeline
- Integration of all modules
- Lip visibility detector
- Automatic AV → Audio-only switching
- Speaker tracking
- Interactive speaker selection
- Bounding box rendering
- User interface
- End-to-end testing

This demonstrates substantial original work even though some core models are pretrained.

Speech separation

This is the most difficult part of the project.

I would **not** train a speech separation network from scratch.

Instead:

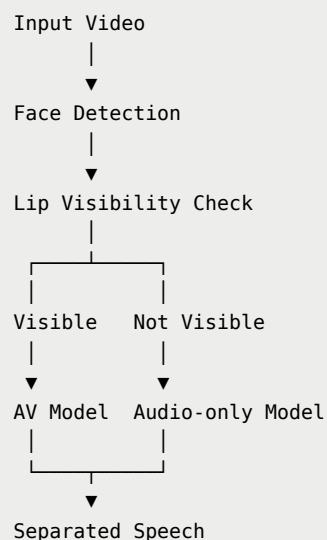
- Use an existing pretrained audio-visual speech separation model.
- Fine-tune it only if necessary.
- Otherwise, use it as the backbone and focus on the overall system.

That keeps the project feasible.

Missing visual cues (your novelty)

This is the feature that distinguishes your project.

Instead of reproducing the full RAVSS architecture, implement a lightweight controller such as:



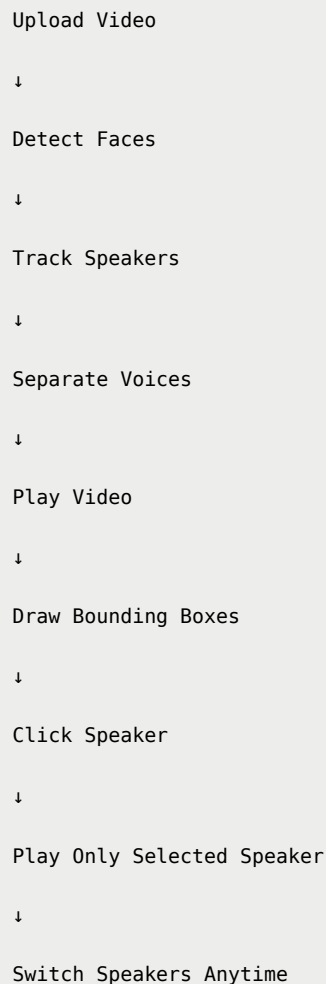
The visibility check can use practical signals such as:

- Face detection confidence
- Landmark confidence
- Mouth landmark visibility
- Occlusion percentage
- Number of consecutive missing frames

This achieves the same functional goal as RAVSS without implementing its full research architecture.

User interface (your biggest contribution)

Your UI is more advanced than what many research implementations provide.



```
graph TD; A[Upload Video] --> B[Detect Faces]; B --> C[Track Speakers]; C --> D[Separate Voices]; D --> E[Play Video]; E --> F[Draw Bounding Boxes]; F --> G[Click Speaker]; G --> H[Play Only Selected Speaker]; H --> I[Switch Speakers Anytime];
```

Upload Video

↓

Detect Faces

↓

Track Speakers

↓

Separate Voices

↓

Play Video

↓

Draw Bounding Boxes

↓

Click Speaker

↓

Play Only Selected Speaker

↓

Switch Speakers Anytime

This makes the project feel like a complete application rather than just a model inference script.

Why this is a good final-year project

This approach demonstrates that you:

- understand current research,
- select appropriate pretrained models,
- design a complete system architecture,
- integrate multiple AI components,
- implement a practical fallback mechanism,

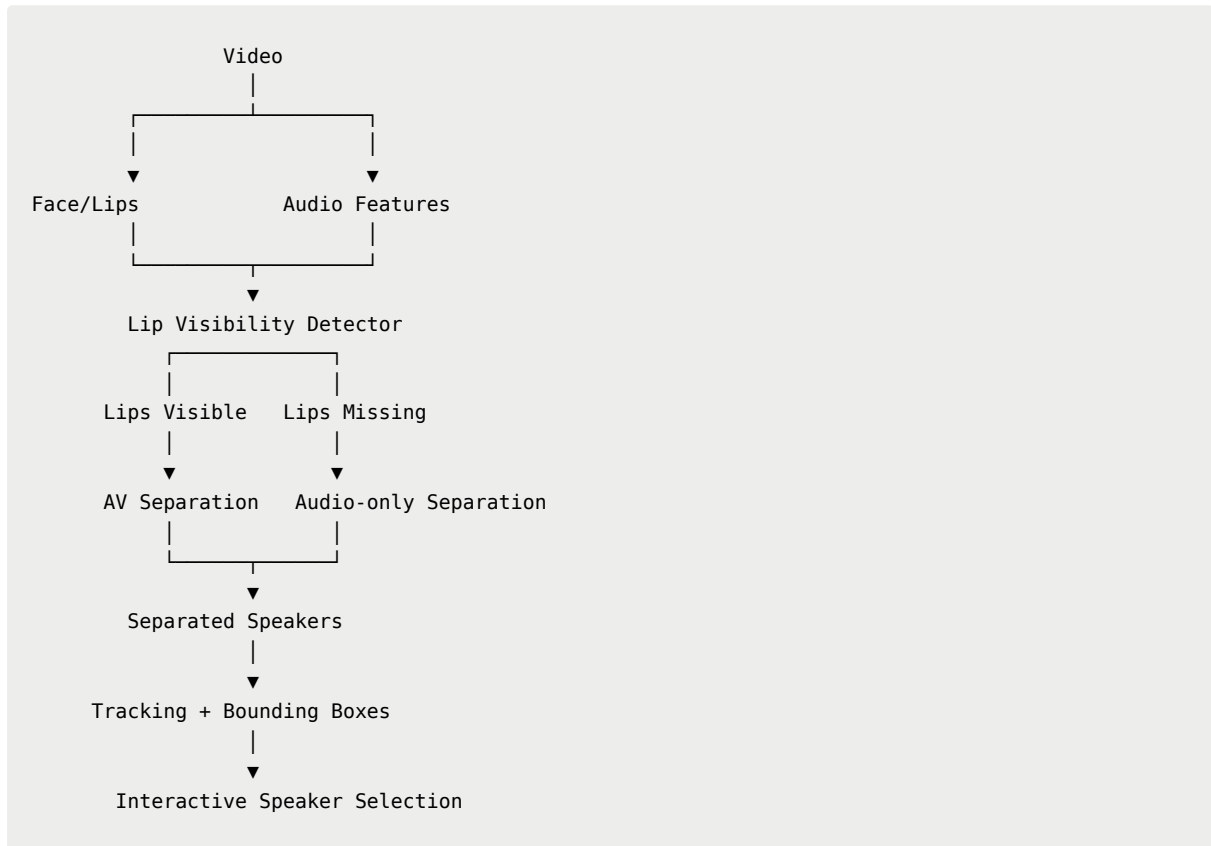
- develop an interactive application.

That is a stronger engineering project than reimplementing a single paper.

One recommendation

I would make one small architectural improvement:

Instead of relying on **one** AV model, use two specialized pretrained models:



This is more robust than forcing a single model to handle both scenarios. When visual information is unavailable, the system explicitly switches to a dedicated audio-only separation model, making the fallback behavior clear and reliable.

Overall, I would rate this implementation strategy **9.5/10** for a final-year project. It aligns with your literature survey and proposed methodology, is technically achievable within a student project, leverages state-of-the-art open-source components appropriately, and still contains enough original system design and integration work to demonstrate meaningful engineering effort.